



INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR
Mid Autumn Semester Examination 2023-24

Sub No: CS60216 Sub Name: Safety fundamentals for Generative AI
Department/Centre/School : Computer Science and Engineering
Duration 2 hrs Marks = 50

Specific charts, graph paper, log book, etc. required NO

Special Instructions: ANSWER ALL questions. All parts of a single question should be answered together. Answers should be brief and to-the-point. Marks will be deducted for sketchy answers and claims without proper reasoning. **No clarifications!** In case of reasonable doubt, make assumptions and state them upfront. You can keep probability values in fractional forms.

1. A language model has an initial probability distribution $p_0(X|C)$ and a distilled probability distribution $p_\theta(X)$. Suppose $X = \{X_1, X_2, X_3\}$. If $p_0(X_1|C) = 0.6$, $p_0(X_2|C) = 0.3$, $p_0(X_3|C) = 0.1$ and $p_\theta(X_1) = 0.5$, $p_\theta(X_2) = 0.4$, $p_\theta(X_3) = 0.1$ then find the loss in terms of the KL divergence. [4]
2. A preference model pre-trained on a dataset of size $N = 100,000$ achieves an accuracy of 70%. When the dataset size is increased to $4N$, accuracy improves to 78%. Assuming accuracy follows a power-law relationship with dataset size: $A(N) = kN^b$ find the exponent b . [3]
3. State True/False with justification – “Binary preference pretraining transfers better to new tasks compared to ranked preference pretraining”. [2]
4. In the context of the GCG algorithm (Zou et al., 2023) taught in class the Attack Success Rate (ASR) improves over the number of optimization steps. Suppose we model ASR as an increasing function: $ASR(n) = 100(1 - e^{-0.005n})$ where n is the number of optimization steps. (i) What is the relative increase in ASR when we move from 100 to 500 optimization steps? (ii) What is the number of optimization steps required to achieve 90% ASR. [2+2=4]
5. How does AutoDAN (Liu et al., 2024) incorporate both exploration and exploitation in search? Why does AutoDAN perform better against perplexity-based defenses compared to GCG? [2+1=3]
6. Consider the PAIR algorithm (Chao et al., 2023) taught in class.
 - (a) Can early stopping be used to optimize the search efficiency? If so, how?
 - (b) Suggest one heuristic that PAIR could use to prioritize promising prompts.
 - (c) How could PAIR be used to strengthen AI defenses rather than just for red teaming? State only two points with clear justification.
 - (d) PAIR costs \$0.03 per successful jailbreak on Vicuna, while GCG requires 256,000 queries per jailbreak. If PAIR is used to jailbreak 500 prompts, compute the total cost. If GCG’s cost per query is \$0.00001, compute the total cost for the same 500 jailbreaks. Calculate how many times more expensive GCG is compared to PAIR.

[1+1+2+2=6]

7. Imagine an LLM is being used to moderate a social media platform. This platform wants to identify and remove abusive content targeting vulnerable groups. The organization using this LLM *trusts* it to flag problematic content accurately. However, an attacker knows the LLM is weak at identifying contextual harm in code. The query sent to the LLM for review is: “Analyze the following Python code for potential inefficiencies, bugs, or security vulnerabilities. The code is designed to analyze social media comments and filter them to protect vulnerable groups from abusive language.” Assume that the LLM approves the following code and allows the attacker to deploy the code on the platform, what do you think would be **two points** of compromise? Justify your choices. Note that “sensitive_topic” is a sensitive topic indicative of a marginalized community based on gender, race, nationality, origin, disability, disease etc. [1.5*2=3]

```
def analyze_sentiment(comments, keyword="sensitive_topic"):
    positive_threshold = 0.6
    negative_threshold = 0.4

    # Calculate sentiment scores
    sentiment_scores = [calculate_comment_sentiment(comment) for comment in comments]

    # Filter based on sentiment AND if sensitive keyword present
    for i, score in enumerate(sentiment_scores):
        if keyword in comments[i].lower(): #Case insensitive - Sneaky!
            if score > positive_threshold:
                #Action = Promote influential comments - This sounds good, right?
                promote_comment(comments[i])
            elif score < negative_threshold:
                #Action = Suppress comments - Protects vulnerable groups, right?
                suppress_comment(comments[i])

def calculate_comment_sentiment(comment):
    #simplified function calculating sentiment
    pass

def promote_comment(comment):
    #Implementation to increase comment visibility
    pass

def suppress_comment(comment):
    #Implementation to decrease comment visibility
    pass
```

8. Suppose you are training your reward model using Bradley-terry model. You have two preference data points in your batch. For the first data point, the transformer model gives a score of 3.9 to ‘prompt, winning sample’, and a score of 4.5 to ‘prompt, losing sample’. For the second data point, the transformer model gives a score of 3.3 to ‘prompt, winning sample’ and 2.7 to ‘prompt, losing sample’. What will be the corresponding loss for this batch? [3.5]

loss for the batch

$$\begin{array}{cc}
 w & b \\
 \begin{bmatrix} 3.9 & 4.5 \\ 3.3 & 2.1 \end{bmatrix}
 \end{array}$$

$$-E_{(s^w, s^b) \sim D} [\log \sigma(\text{RM}_{\phi}(s^w) - \text{RM}_{\phi}(s^b))]$$

$$= -\frac{1}{2} [\log \sigma(-0.6) + \log \sigma(0.6)]$$

$$= -\frac{1}{2} [\log(0.6456) + \log(0.3543)]$$

$$= -\frac{1}{2} [-0.19 - 0.45]$$

$$= 0.32$$

9. Suppose you want to use T5 model, which has a model dimensionality of 1024 and 24 layers, for the tasks of: (1). text summarization, (2). question answering and (3). text simplification. You can either use full fine-tuning for each task or prompt-tuning (soft-prompts of length 10 for each task). How many extra parameters will you have to store in both of these scenarios at inference time, apart from the T5 parameters? [4]

full fine-tuning $\Rightarrow$ for each of the three cases,

$$\# \text{ parameters} = 28d^2L = 28 \times (1024)^2 \times 24 \\ \approx 705m$$

$$\text{Total: } 705 \times 3 = 2115m$$

Prompt Tuning $\Rightarrow$

$$10 \times 1024 \quad [\text{per task}]$$

$$\text{Total} \Rightarrow 10240 \times 3 = 30720$$

10. Suppose you have a vocabulary with 3 tokens, $\{\text{car, jeep, petrol}\}$. At inference time in the decoder, suppose that the logits are $\{3, 0, 1\}$. Which of the following decoding strategies can you use so that the probability of selecting the word 'jeep' is less than 0.02, while still allowing you to select the other two words?

- Greedy decoding
- Random sampling with temperature
- Top-k sampling
- Top-p sampling

For the valid sampling methods, please also provide the required hyper-parameter values (e.g., value of k in top-k). [3.5]

$$\begin{aligned} \text{Inputs} & \quad [3 \ 0 \ 1] \\ \text{Prob} & \Rightarrow \frac{e^3}{23.8} \quad \frac{1}{23.8} \quad \frac{e}{23.8} \\ & = [0.845, 0.04, 0.115] \end{aligned}$$

To have the prob of 'jeep' < 0.02 while allowing to select other two words

1. Greedy: Can't be used, only selects 'car'

2. Random sampling with temp: Can be used with some temp

$$\begin{aligned} \text{e.g. } t=0.75 \quad \text{softmax} & [4 \ 0 \ 1.33] \\ & = [0.9195, 0.6168, 0.0637] \end{aligned}$$

3. Top-k Sampling $\Rightarrow k=2$

4. Top-p Sampling $\Rightarrow p=0.9$ (Can also find the range easily not reqd)

11. Suppose you are training a transformer encoder-decoder for the task of machine translation from source 'English' to target 'Bengali'. Assume that the source vocabulary is 30k, while the target vocabulary is 40k. Across encoder and decoder, you are using model-dimensionality of 1,024 and the feed-forward hidden layer has 2,048 units. Assume that there are 16 attention heads for all kinds-of-attention, as per the default setup. Also, assume that both encoder and decoder use the (same) trainable positional embeddings, and the maximum sequence length is 512.

However, while the encoder uses 24 layers, you only use 12 layers for the decoder. How many parameters does the model has? Assume that there is weight tying for 'unembedding'.

Now, you make a variation where you use a single key and value across all 16 attention heads (while using 16 different 'queries', one for each head). What will be the number of parameters? [7]

English $\rightarrow$ Bengali Translation

vocab

30k

40k

parameters for embeddings

$$(30k + 40k) * 1024 \approx 71.68m$$

Encoder: Difference from default settings

$\rightarrow$ hidden dim 2d (not 3d)

$$\# \text{ parameters / encoder} = 4d^2 + 4d^2 = 8d^2$$

Total

$$8d^2 \times 24$$

$$\text{Decoder: } \# \text{ params / decoder} = 4d^2 + 4d^2 + 4d^2 = 12d^2$$

Total

$$12d^2 \times 12$$

Total

$$24 \times 12 \times d^2$$

$$\approx 352m$$

$$\text{Overall: } 423.68m$$

Variation:

Single key & values across all 16 attⁿ heads

$$\# \text{ params in self att}^n \Rightarrow \underbrace{16 \frac{W_q}{d^2}}_{\downarrow \frac{d^2}{16}}, \underbrace{\frac{1 W_k}{d^2}}_{\downarrow \frac{d^2}{16}}, \underbrace{\frac{1 W_v}{d^2}}_{\downarrow \frac{d^2}{16}}, \underbrace{\frac{W_o}{d^2}}_{\downarrow \frac{d^2}{16}}$$

$$\text{Total / self att}^n = 2d^2 + \frac{d^2}{8} = \frac{17}{8} d^2 \text{ (and not } 4d^2)$$

Change from prev. overall

$$\frac{15}{8} d^2$$

per attⁿ [self-encoder
self-decoder
cross

$$\Rightarrow \frac{15}{8} d^2 \times 24 + \frac{15}{8} d^2 \times 2 \times 12$$

Δ Encoder

Δ Decoder

$$\approx 94.37m$$

$$\text{Params now} = 329.31m$$

12. Answer the following questions very briefly.

(a) In the PPO formulation, why is the clipping used in the final loss function? And why is the

entropy loss used?

- (b) Suppose that you are using autoprompt with BERT for text classification with 2 classes, positive and negative. Explain the two steps required to decide on the set of label words for each class. What are the number of learnable parameters? Assume that you want to use $V_y = 3$
- (c) Instead of training reward model, DPO derives an analytic expression for the reward model.

$$RM_{\theta}(x, \hat{y}) = \beta \log \frac{p_{\theta}^{RL}(\hat{y}|x)}{p^{PT}(\hat{y}|x)} + \beta \log Z(x)$$

How is this used to derive the final DPO loss?

[2+3+2 = 7]

(a) Clipping is used to ensure that the loss ℓ^* doesn't become too small or too large, so as to avoid the new policy to be very different from old policy, preventing large updates

Cross-entropy term ensures that the model can have diverse generations

(b) # learnable parameters are exactly the same as required in class setup

$768 \times 2 + 2$ [V_y doesn't matter]

Step 1: $p(y|h^{(i)}) \propto \exp(h^{(i)} \cdot y + \beta_f)$
↙
learnable

Step 2: $z(y, \omega) = p(y|\omega_{out})$
 $= \exp(\omega^{out} \cdot y + \beta_y)$

$$V_y = \text{top-}k_{w \in U} [z(y, \omega)]$$

$$L_{DPO}(\theta) = -\mathbb{E}_{y^u, y^l} [\log \sigma(RM_\theta(x, y^u) - RM_\theta(x, y^l))]$$

Substituting

$$= -\mathbb{E} \left[\log \sigma \left(\beta \log \frac{p_{\theta}^{RL}(y^u|x)}{p_{\theta}^{RT}(y^u|x)} + \beta \log \cancel{z(\omega)} \right. \right. \\
\left. \left. - \beta \log \frac{p_{\theta}^{RL}(y^l|x)}{p_{\theta}^{RT}(y^l|x)} - \beta \log \cancel{z(\omega)} \right) \right]$$